TALLER SEMANA 3

IMPLEMENTACIÓN PATRON SINGLETON Y FACTORY

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

1. Estructura del proyecto Enerlink

```
└── enerlink
    └── configuracion
        ├── PredictionEngine.java
    └── usuario
        └── aplicacion
            └── servicio
                ├── UserService.java
        └── dominio
            └── factory
                ├── ConsumerUserFactory.java
                ├── MixedUserFactory.java
                ├── ProducerUserFactory.java
                ├── UserFactory.java
            └── modelo
                ├── ConsumerUser.java
                ├── MixedUser.java
                ├── ProducerUser.java
                ├── User.java
            └── puerto
                ├── UserRepositoryPort.java
        └── infraestructura
            └── controlador
                ├── UserController.java
                ├── UserRequest.java
            └── persistencia
                ├── UserEntity.java
                ├── UserJpaRepository.java
                ├── UserRepositoryAdapter.java
    └── EnerlinkApplication.java
```

2. Arquitectura Hexagonal Aplicada en Enerlink

El proyecto está organizado bajo el modelo de Arquitectura Hexagonal (Ports and Adapters). Esto significa que el núcleo del sistema se encuentra en el dominio y que todo elemento externo accede a él a través de contratos bien definidos (puertos).

En la estructura del proyecto se observa claramente esta separación:

- El paquete dominio contiene las entidades (User y sus subclases), las fábricas y el puerto UserRepositoryPort.

- El paquete aplicación contiene el servicio UserService, que coordina los casos de uso.

- El paquete infraestructura contiene el controlador REST y el adaptador JPA que implementa el puerto del dominio.

La característica central de esta arquitectura es la dirección de dependencias: la infraestructura depende del dominio, pero el dominio no depende de la infraestructura. Por ejemplo, UserRepositoryAdapter implementa UserRepositoryPort, pero el dominio no conoce JPA ni Spring.

Este diseño permite cambiar detalles técnicos sin modificar la lógica de negocio. Si se reemplazara JPA por otra tecnología de persistencia, el dominio y el servicio no tendrían que modificarse, ya que dependen de una abstracción.

Desde el punto de vista de SOLID:

- Se aplica el principio de inversión de dependencias (DIP), ya que las capas superiores no dependen de implementaciones concretas sino de interfaces.

- Se respeta el principio de responsabilidad única (SRP), porque cada capa cumple una función específica: el dominio modela el negocio, la aplicación coordina y la infraestructura conecta con el exterior.

- Se facilita el principio abierto/cerrado (OCP), ya que pueden agregarse nuevas implementaciones sin alterar el núcleo.

En términos de Clean Code, la separación por paquetes refleja límites claros de responsabilidad, evitando mezclas entre lógica de negocio y detalles técnicos.

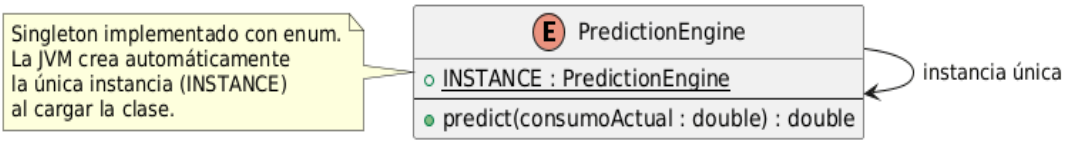
3. Patrón Singleton — PredictionEngine

3.1 Descripción del patrón

El patrón Singleton garantiza que exista una única instancia de una clase durante el ciclo de vida de la aplicación y que dicha instancia sea accesible de manera controlada.

En este proyecto, el patrón se implementó mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*.

3.2 Diseño UML



3.3 Implementación

```java
package com.enerlink.enerlink.configuracion;

import java.util.concurrent.ThreadLocalRandom;

/**
 * PATRÓN SINGLETON — Implementación usando Enum (Joshua Bloch)
 *
 * Esta es la forma recomendada en Effective Java.
 *
 * Ventajas:
 * - Thread-safe automáticamente
 * - Protección contra serialización
 * - Protección contra reflexión
 * - Implementación simple y robusta
 *
 * No depende de Spring.
 */

public enum PredictionEngine {

    INSTANCE; // Única instancia garantizada por la JVM

    /**
     * Método de negocio: predice el consumo futuro.
     *
     * @param consumoActual consumo actual en kWh
     * @return predicción de consumo futuro
     */
    public double predict(double consumoActual) {

        double factorBase = 1.15;

        // Variación aleatoria entre -5% y +5%
        double variacion = ThreadLocalRandom.current().nextDouble(-0.05, 0.05);

        double factorFinal = factorBase + variacion;

        return consumoActual * factorFinal;
    }
}
```

3.4 Justificación de la implementación

La clase PredictionEngine implementa el patrón **Singleton** mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*. Esta implementación garantiza que la **JVM cree una única instancia** de PredictionEngine, evitando la posibilidad de generar instancias adicionales mediante reflexión o serialización.

El acceso a la instancia se realiza de manera simple:

PredictionEngine.INSTANCE

No existe constructor público ni método getInstance(), ya que el propio mecanismo del enum controla la instanciación.

Desde el punto de vista técnico:

- La clase no mantiene estado mutable; el método predict utiliza únicamente variables locales, lo que asegura **seguridad en entornos multihilo**.
- Cumple con el principio de **Responsabilidad Única**, ya que su única función es calcular predicciones de consumo.
- No introduce **acoplamientos innecesarios**, ya que no depende de Spring ni de otras capas del proyecto.

Desde el punto de vista de negocio en Enerlink:

- PredictionEngine es un **servicio compartido sin estado**, utilizado por múltiples componentes del sistema, como UserService.
- Tener una única instancia asegura que las predicciones sean consistentes en todo el proyecto, evitando discrepancias entre diferentes cálculos.
- Reduce el consumo de memoria y simplifica la lógica, al eliminar la necesidad de crear, inyectar o gestionar múltiples instancias.
- Facilita la mantenibilidad y la extensión del proyecto, ya que cualquier cambio en la lógica de predicción se realiza en un solo lugar.

En resumen, la implementación **cumple los objetivos del patrón Singleton**, garantiza consistencia, eficiencia y simplicidad, y aporta **beneficios claros tanto desde el diseño técnico como desde la operación del sistema**.

3.5 Prueba de funcionamiento

```java
package com.enerlink.enerlink.configuracion;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class SingletonDemoRunner implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        PredictionEngine engine1 = PredictionEngine.INSTANCE;
        PredictionEngine engine2 = PredictionEngine.INSTANCE;
        PredictionEngine engine3 = PredictionEngine.INSTANCE;
        PredictionEngine engine4 = PredictionEngine.INSTANCE;

        System.out.println("==========================================");
        System.out.println("          DEMO PATRÓN SINGLETON           ");
        System.out.println("==========================================\n");

        System.out.println("Verificación de que todas las instancias son la misma:");
        System.out.println("------------------------------------------");
        System.out.printf("engine1 == engine2 ? %b%n", engine1 == engine2);
        System.out.printf("engine1 == engine3 ? %b%n", engine1 == engine3);
        System.out.printf("engine1 == engine4 ? %b%n", engine1 == engine4);

        System.out.println("\nUso del método predict():");
        System.out.println("------------------------------------------");
        double consumoActual = 100.0;
        double prediccion = engine1.predict(consumoActual);
        System.out.printf("Consumo actual: %.2f kWh%n", consumoActual);
        System.out.printf("Predicción de consumo: %.2f kWh%n", prediccion);

        System.out.println("\n==========================================");
        System.out.println("          FIN DE LA DEMO SINGLETON        ");
        System.out.println("==========================================\n");

```

```
==========================================
          DEMO PATRÓN SINGLETON
==========================================

Verificación de que todas las instancias son la misma:
------------------------------------------
engine1 == engine2 ? true
engine1 == engine3 ? true
engine1 == engine4 ? true

Uso del método predict():
------------------------------------------
Consumo actual: 100,00 kWh
Predicción de consumo: 111,79 kWh

==========================================
          FIN DE LA DEMO SINGLETON
==========================================
```

Como se puede observar, todas las variables creadas (engine 1, 2, 3…) apuntan a una misma instancia y permite utilizar su método predict(). Lo que significa que el patrón singleton está funcionando correctamente dentro del proyecto.

3.6 Aclaración sobre la implementación actual

Por motivos de **practicidad y enfoque académico**, la clase PredictionEngine no implementa un algoritmo real de predicción de consumo. El objetivo principal de la implementación actual es **demostrar correctamente el patrón Singleton**, garantizando:

- Instancia única compartida.
- Seguridad en entornos multihilo.
- Acceso uniforme desde cualquier parte del sistema.

A futuro, se planea reemplazar el método predict por un **algoritmo real de predicción**, capaz de analizar más variables y atributos relevantes para el consumo energético. En ese momento, el PredictionEngine podrá evolucionar para:

- Manejar más parámetros de entrada y modelos complejos.
- Mantener la instancia única, asegurando consistencia en todas las llamadas.
- Adaptarse a necesidades de cálculo más avanzadas sin romper la arquitectura ni el patrón Singleton.

La versión actual es **prototipo funcional del patrón**, mientras que la lógica de negocio del cálculo de predicciones será ampliada en versiones futuras del sistema.

4. Patrón Factory Method

4.1 Descripción del patrón

El **Factory Method** es un patrón de diseño creacional que define un contrato para crear objetos, pero permite que las subclases decidan **qué clase concreta instanciar**.

En Enerlink, este patrón se utiliza para que el UserService pueda **crear distintos tipos de usuarios** (ProducerUser, ConsumerUser, MixedUser) sin depender directamente de sus clases concretas. Esto asegura que la capa de aplicación permanezca **desacoplada de los detalles de implementación del dominio**, facilitando mantenimiento y extensibilidad.

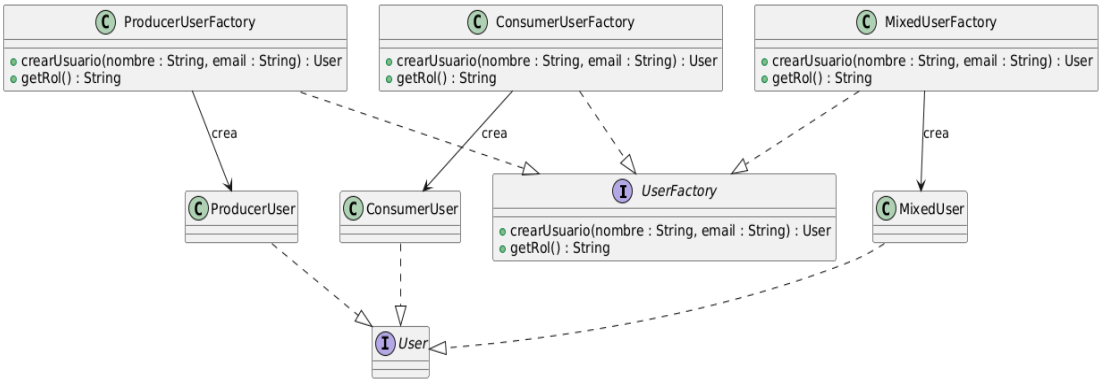
Desde el punto de vista de diseño, el patrón favorece:

- **Open/Closed Principle**: nuevos tipos de usuario se pueden agregar sin modificar UserService.

- **Single Responsibility Principle**: cada factory se encarga únicamente de crear un tipo de usuario.

- **Dependency Inversion Principle**: el servicio depende de la abstracción UserFactory y no de implementaciones concretas.

4.2 Diagrama UML



4.4 Implementación

Clase abstracta User

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Clase abstracta del dominio. No depende de Spring.
 * Define el contrato común de cualquier tipo de usuario en Enerlink.
 */
public abstract class User {

    protected Long id;
    protected String nombre;
    protected String email;
    protected String rol; // "PRODUCER", "CONSUMER" o "MIXED"

    public User(String nombre, String email, String rol) {
        this.nombre = nombre;
        this.email = email;
        this.rol   = rol;
    }

    public String describir() {
        return "Usuario: " + nombre + " con rol: " + rol;
    }

    // Getters
    public Long getId()       { return id; }
    public String getNombre() { return nombre; }
    public String getEmail()  { return email; }
    public String getRol()    { return rol; }

    public void setId(Long id) { this.id = id; }
}
```

Clase concreta ProducerUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario productor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class ProducerUser extends User {

    public ProducerUser(String nombre, String email) {
        super(nombre, email, "PRODUCER");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + " y produzco energía en Enerlink.";
    }
}
```

Clase concreta ConsumerUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario productor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class ProducerUser extends User {

    public ProducerUser(String nombre, String email) {
        super(nombre, email, "PRODUCER");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + " y produzco energía en Enerlink.";
    }
}
```

Clase concreta MixedUser

```java
package com.enerlink.enerlink.usuario.dominio.modelo;

/**
 * Usuario consumidor de energía.
 * Producto concreto del patrón Factory Method.
 */
public class MixedUser extends User {

    public MixedUser(String nombre, String email) {
        super(nombre, email, "MIXED");
    }

    @Override
    public String describir() {
        return "Soy " + nombre + ", Produzco y consumo energía en Enerlink.";
    }
}
```

Clase abstracta UserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * FACTORY METHOD CLÁSICO
 *
 * Clase abstracta que define el contrato de cualquier Factory de usuarios.
 * Cada subclase concreta decide qué tipo de usuario crear.
 */
public abstract class UserFactory {

    /**
     * Método factory abstracto: cada subclase concreta implementa la creación de su usuario.
     *
     * @param nombre nombre del usuario
     * @param email  correo del usuario
     * @return instancia concreta de User
     */
    public abstract User crearUsuario(String nombre, String email);

}
```

Clase concreta ProducerUserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.ProducerUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios productores de energía.
 * Producto concreto del patrón Factory Method.
 */
@Component("PRODUCER")
public class ProducerUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new ProducerUser(nombre, email);
    }
}
```

Clase concreta ConsumerUserFactory

```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.ConsumerUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios consumidores de energía.
 * Producto concreto del patrón Factory Method.
 */
@Component("CONSUMER")
public class ConsumerUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new ConsumerUser(nombre, email);
    }
}
```

Clase concreta MixedUserFactory

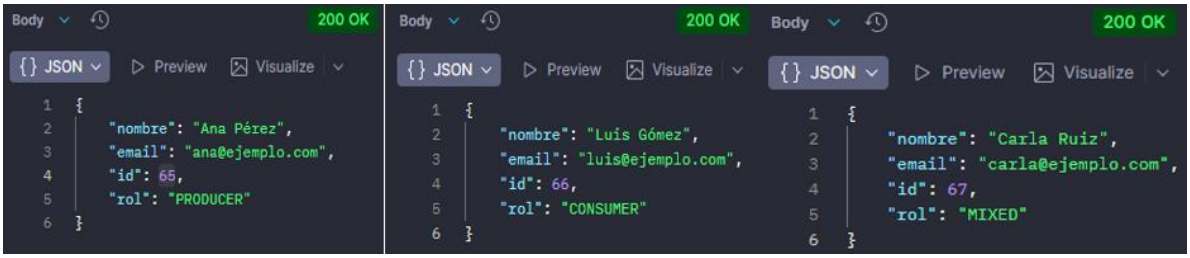
```java
package com.enerlink.enerlink.usuario.dominio.factory;

import org.springframework.stereotype.Component;

import com.enerlink.enerlink.usuario.dominio.modelo.MixedUser;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

/**
 * Factory concreto que crea usuarios mixtos (producen y consumen energía).
 * Producto concreto del patrón Factory Method.
 */
@Component("MIXED")
public class MixedUserFactory extends UserFactory {

    @Override
    public User crearUsuario(String nombre, String email) {
        return new MixedUser(nombre, email);
    }
}
```

4.4 Prueba de funcionamiento

Creación de usuarios por rol:

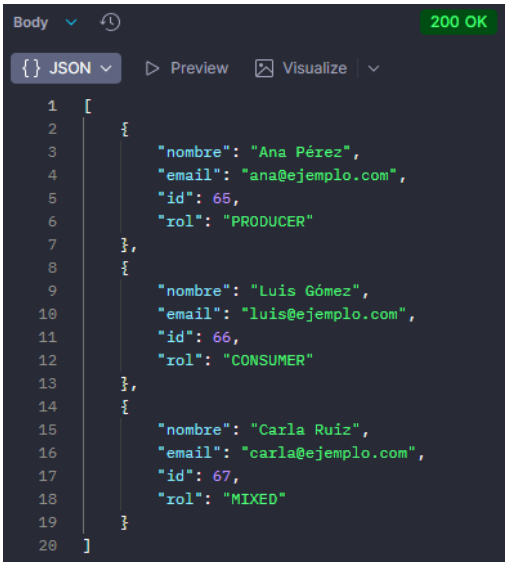
Se usó el endpoint **POST /api/usuarios** para crear un usuario de cada tipo: **Producer**, **Consumer** y **Mixed**:

Rol	JSON enviado
Producer	{ "nombre": "Ana Pérez", "email": "ana@ejemplo.com", "rol": "PRODUCER" }
Consumer	{ "nombre": "Luis Gómez", "email": "luis@ejemplo.com", "rol": "CONSUMER" }
Mixed	{ "nombre": "Carla Ruiz", "email": "carla@ejemplo.com", "rol": "MIXED" }



Listar todos los usuarios:

Para comprobar que los usuarios se crearon correctamente y que cada instancia mantiene su comportamiento polimórfico, se llamó al endpoint **GET /api/usuarios**.



Como se puede observar, se logró crear y listar usuarios con distintos roles, demostrando que el patrón Factory maneja correctamente la creación de los distintos tipos de usuarios que existen.

4.5 Justificación de la implementación

Técnicamente:

- UserService nunca hace new ProducerUser(...) ni new ConsumerUser(...).
- La creación de usuarios se delega a la factory correspondiente según el rol, garantizando que el servicio no dependa de clases concretas.
- Si se requiere un nuevo tipo de usuario, solo hay que agregar una nueva factory y registrarla en el mapa de factories, sin modificar UserService.

Desde la perspectiva del negocio en Enerlink:

- Cada usuario puede tener un comportamiento propio (por ejemplo, describir()).
- La fábrica asegura que el UserService reciba siempre el tipo correcto, garantizando consistencia y mantenimiento fácil.
- Reduce errores al centralizar la lógica de creación y mejora la capacidad de escalar el sistema.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de **abstracciones** (UserFactory) y no de clases concretas.
- Cada factory tiene **una única responsabilidad**: crear un tipo de usuario concreto.
- El sistema es **abierto a extensiones y cerrado a modificaciones**, cumpliendo Open/Closed Principle.

4.6 Aclaración sobre la implementación actual

Actualmente, las factories devuelven usuarios con datos básicos (nombre, email, rol) y su método describir(). Esto es suficiente para demostrar el patrón Factory Method sin introducir lógica de negocio compleja en la creación de usuarios.

A futuro, si se requiere agregar configuraciones iniciales adicionales o atributos dinámicos en los usuarios, las factories podrán evolucionar para incluirlos sin afectar UserService ni romper el patrón.

Referencias Bibliográficas

Refactoring.Guru. (s. f.). *Singleton*. https://refactoring.guru/es/design-patterns/singleton

Refactoring.Guru. (s. f.). *Factory Method*. https://refactoring.guru/es/design-patterns/factory-method

SourceMaking. (s. f.). *Factory Method design pattern*. https://sourcemaking.com/design_patterns/factory_method

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. http://www.cse.hcmut.edu.vn/~thai/books/Design_Patterns_Elements_of_Reusable_Object-Oriented_Software.pdf

Freeman, E., & Robson, E. (2004). *Head first design patterns*. O'Reilly Media. https://www.staff.ces.funai.edu.ng/textbook-solutions/Resources/HomePages/Head_First_Design_Patterns_Eric_Freeman.pdf